

Цель работы: получить практический опыт базовой установки PostgreSQL, настройки памяти и использования ядер CPU, настройки сетевого доступа, управления пользователями и ролями, а также разных видов бэкапа (логический и физический).

i

А нужно ли это девопсу?

В стартапах и средних компаниях девопс обычно заведует всей инфраструктурой, в том числе базами данных, поэтому это могут требовать в вакансиях. В бигтехах обычно для этого есть отдельный специалист Database Administrator — DBA. В любом случае, на базовом уровне девопс должен знать базы данных.

Теоретическая справка

PostgreSQL (или Postgres) — объектно-реляционная система управления базами данных с открытым исходным кодом. Она была создана в 1986 году.

PostgreSQL сочетает принципы реляционной модели с современными средствами работы с объектами и пользовательскими типами данных. Область применения PostgreSQL широкая. Она применяется как в бизнесе, так и в научной среде. Система поддерживает стандарты SQL.

Ключевое преимущество PostgreSQL — расширяемость: разработчик может создавать собственные типы данных, функции и операторы, адаптируя систему под конкретные задачи. Отсутствие лицензионных ограничений делает PostgreSQL независимой от поставщиков и привлекательной для внедрения в образовательных и производственных средах.

Актуальная стабильная версия PostgreSQL на февраль 2026 года — 18.2. Официальный сайт проекта: [postgresql.org](https://www.postgresql.org).

Основные возможности:

- Поддержка широкого набора типов данных — числовых, символьных, логических, временных, двоичных, JSON, UUID и других.
- Гибкие механизмы индексации (B-tree, GIN, GiST и др.), ускоряющие поиск и обработку данных.
- Полная поддержка транзакций с соблюдением принципов ACID (атомарность, согласованность, изолированность, долговечность).
- Многоверсионное управление параллелизмом (MVCC), позволяющее выполнять конкурентные операции с минимумом блокировок и с гарантией целостности данных.
- Поддержка материализованных представлений — инструмент для хранения результатов сложных аналитических запросов.
- Реализация партиционирования таблиц, повышающая производительность при работе с большими объёмами данных.

Архитектурные компоненты PostgreSQL:

- Процесс postmaster — это основной родительский процесс PostgreSQL. Он запускается первым при старте сервера и отвечает за управление всеми служебными процессами и клиентскими подключениями. Он следит за входящими подключениями и создаёт дочерние процессы для работы с клиентами и служебными задачами. В современных версиях PostgreSQL в выводе ps он называется postgres, но по сути это тот же postmaster (для этого процесса родительский PID = 1).
- Backend-процессы — обслуживают клиентские запросы и выполняют SQL — процессы postgres (по одному на каждое активное соединение)
- Background-процессы — выполняют фоновое обслуживание (автоочистка, WAL, контрольные точки) — процессы writer, wal writer, checkpointer, autovacuum launcher, archiver
- Shared Buffers — область общей памяти для кэширования данных страниц — разделяемая память (shared memory segment, видимый через ipcs -m)
- WAL (Write-Ahead Log) — журнал предзаписи для обеспечения надёжности восстановления — файлы-сегменты в подкаталоге pg_wal каталога данных

Пример вывода `ps -ef | grep postgres`:

```
postgres 1234      1  0 10:00 ?        00:00:00 postgres
postgres 1250    1234  0 10:00 ?        00:00:01 postgres: checkpointer process
postgres 1251    1234  0 10:00 ?        00:00:01 postgres: background writer process
postgres 1252    1234  0 10:00 ?        00:00:01 postgres: wal writer process
postgres 1253    1234  0 10:00 ?        00:00:01 postgres: autovacuum launcher
postgres 1254    1234  0 10:00 ?        00:00:01 postgres: stats collector
postgres 1300    1234  0 10:05 ?        00:00:00 postgres: postgres user connection
user      1350    1300  0 10:07 pts/0    00:00:00 grep --color=auto postgres
```

Конфигурационные файлы

После установки PostgreSQL основные настройки системы хранятся в каталоге данных (обычно `/var/lib/postgresql/<версия>/main/` или `/var/lib/pgsql/data/`) или каталоге конфигурационных файлов ОС (`/etc/postgresql/<версия>/main/`).

Основные файлы конфигурации:

- `postgresql.conf` — главный конфигурационный файл сервера. Здесь задаются параметры производительности, подключения, логирования и использования памяти.
- `pg_hba.conf` — файл управления доступом (Host-Based Authentication), в котором определяется, какие пользователи, хосты и базы данных могут подключаться и какими методами авторизации.

- `pg_ident.conf` — используется для сопоставления системных имён пользователей с именами PostgreSQL при определённых режимах аутентификации.

Изменения в этих файлах требуют перезапуска или перезагрузки конфигурации сервера для применения параметров.

Основные параметры настройки

PostgreSQL использует несколько ключевых параметров памяти: `shared_buffers` (общий буферный кеш), `work_mem` (операции сортировки/хэшей на запрос), `maintenance_work_mem` (VACUUM, CREATE INDEX). Их настройка сильно влияет на производительность.

Параллелизм запросов настраивается параметрами `max_worker_processes`, `max_parallel_workers`, `max_parallel_workers_per_gather`, которые завязаны на количестве доступных ядер CPU.

Утилита `psql`

`psql` — это основная консольная утилита для взаимодействия с сервером PostgreSQL. Она позволяет выполнять SQL-запросы, управлять структурами баз данных и просматривать служебную информацию.

Основные приёмы работы:

- Подключение к базе данных: `psql -h <хост> -p <> -U <пользователь> -d <имя_базы>`
- Просмотр списка таблиц: `\dt`
- Вывод списка баз данных: `\l`
- Выполнение SQL-запроса: `SELECT * FROM table_name;`
- Получение справки по системным командам: `?`

Резервное копирование в PostgreSQL

PostgreSQL поддерживает два подхода к резервному копированию — логическое и физическое. Выбор зависит от задач восстановления, объёма данных и требований к скорости.

Логические бэкапы выполняются с помощью утилит `pg_dump` и `pg_dumpall`. При этом создаются переносимые дампы в формате SQL или архивов (custom, tar, directory).

Эти бэкапы предназначены для миграции между версиями PostgreSQL и разными СУБД, выборочного восстановления отдельных таблиц/схем, развёртывание тестовых копий.

Примеры команд:

```
# Бэкап. Одна таблица
pg_dump -h localhost -U postgres -t my-table mydb > table_backup.sql
# Бэкап. Вся база в архив
pg_dump -Fc -U postgres mydb > mydb.backup
pg_dumpall -U postgres > full_dump.sql
#Аналогично восстановление:
psql -U postgres -f table_backup.sql newdb
pg_restore -U postgres -d newdb mydb.backup
```

Физические бэкапы

`pg_basebackup` и копирование файлов WAL создают точную бинарную копию кластера PostgreSQL. Способ предназначен для быстрого восстановления целого кластера, Point-in-time recovery (PITR) с WAL-архивами.

Порядок выполнения работы



Экономьте грант

Выключайте виртуальную машину в случае перерыва в выполнении работы или после завершения.

Подготовка

Выполните `npm install dotenv express multer pg`

В верх файла `SpeedTest.js` добавьте:

```
require('dotenv').config();
const multer = require('multer');
const { Pool } = require('pg');
```

После `Server.use(cors());` добавьте:

```

Server.use(express.urlencoded({ extended: true }));
Server.use(express.json());
const upload = multer();

const pool = new Pool({
  host: process.env.PG_HOST,
  port: process.env.PG_PORT,
  user: process.env.PG_USER,
  password: "postgres",
  database: process.env.PG_DATABASE
});

Server.post('/telemetry', upload.none(), async (req, res) => {
  let requestIP = req.headers['x-forwarded-for'] || req.connection.remoteAddress || req.headers['HTTP_CLIENT_IP'] || req.headers['X-Real-IP'] || req.headers['HTTP_X_FORWARDED_FOR'];
  if (requestIP.substr(0, 7) === "::ffff:") {
    requestIP = requestIP.substr(7)
  }

  const data = req.body;

  try {
    await pool.query(
      `INSERT INTO speedtest_users(
        ip, ispinf, extra, ua, lang, dl, ul, ping, jitter, log
      ) VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)`,
      [
        requestIP || null,
        data.ispinf || null,
        data.extra || null,
        req.headers['user-agent'] || null,
        req.headers['accept-language'] || null,
        data.dl || null,
        data.ul || null,
        data.ping || null,
        data.jitter || null,
        data.log || null
      ]
    );

    res.send({ status: "ok" });
  } catch (err) {
    console.error("Telemetry insert error:", err);
    res.status(500).send({ status: "error" });
  }
});

```

В файле `src/public/speedtest_worker.js` поставьте значения `telemetry_level: 1 url_telemetry: "telemetry"`

Установка

1. Выясните, какая версия PostgreSQL доступна в стандартных репозиториях Ubuntu.
2. Установите PostgreSQL из репозитория дистрибутива (если доступна версия 14 и выше) или официального репозитория (если в репозитория версия младше 14). Добавьте команды в отчет.
3. Убедитесь, что PostgreSQL запущен. Выведите список процессов, относящихся к СУБД. Добавьте команды и консольный вывод в отчет.
4. Убедитесь, что вы можете подключиться локально: `sudo -u postgres psql`. Выведите список баз. Сохраните список баз в отчет.

Настройка

1. Найдите, где расположены файлы `postgresql.conf`, `pg_hba.conf` и каталога данных ("`pgdata`", data-директория), зафиксируйте пути. Сохраните расположение и права на эти файлы в отчет.
2. Определите объём RAM вашей VM и количество ядер CPU. Не смотрите это в панели управления Selectel, подумайте, как это узнать непосредственно из VM.
3. В `postgresql.conf` задайте параметры (если предложенные типовые варианты расчета параметров не подойдут, можно воспользоваться конфигураторами (например <https://www.pgconfig.org>)):
 - Настройте количество процессов для параллельной обработки:
 - `max_worker_processes` = 1,5 × число ядер;
 - `max_parallel_workers` = числу ядер;
 - `max_parallel_workers_per_gather` = 2 или числу ядер / 2 (но не больше общего лимита, задаваемого в `max_worker_processes`)
 - `shared_buffers` ≈ 25% от доступной памяти VM как стартовое значение.
 - `work_mem` на уровне, безопасном с учётом `max_connections`. Соединение иначе Клиентский процесс или бекенд – это отдельный процесс postgres, обслуживающий соединение одного клиента. Каждое активное соединение потребляет ~10MB RAM + `work_mem` × количество операций сортировки/хэширования в запросе.

Формула $work_mem = (RAM_сервера \times 0.25) / (max_connections \times 4)$, где 4 — консервативная оценка количества операций (sort/hash/merge) в запросе.

- *maintenance_work_mem* – память для выполнения процессов обслуживания.
Формула $maintenance_work_mem = (RAM_сервера \times 0.05) / autovacuum_max_workers$
Количество *autovacuum_max_workers* придется определить.

4. Перезапустите PostgreSQL и убедитесь, что параметры применились (запросите новые значения через `SHOW` в `psql`). Сохраните вывод для отчета.
5. Сравните количество процессов PostgreSQL до и после переконфигурирования. Опишите в отчете, что изменилось (если вы обнаружили разницу).

Управление пользователями и ролями

1. Зайдите в *psql* от пользователя *postgres*.
2. Создайте базу данных *debian_kernels*.
3. С помощью SQL скрипта заполните БД, для чего сохраните скрипт в файл и выполните его с помощью команды *psql* на нужной базе данных.

```
CREATE TABLE debian_versions (  
  id          SERIAL PRIMARY KEY,  
  debian_version VARCHAR(20) NOT NULL,  
  codename    VARCHAR(50),  
  kernel_version VARCHAR(20) NOT NULL  
);  
INSERT INTO debian_versions (debian_version, codename, kernel_version) VALUES  
  ('10', 'Buster', '4.19'),  
  ('11', 'Bullseye', '5.10'),  
  ('12', 'Bookworm', '6.1'),  
  ('13', 'Trixie', '6.12');  
  
SELECT * FROM debian_versions ORDER BY id;
```

4. В *psql* создайте пользователя для новой БД, задав ему пароль:

```
CREATE ROLE app_user WITH LOGIN PASSWORD 'ПРИДУМАЙ СЛОЖНЫЙ ПАРОЛЬ!';  
GRANT ALL PRIVILEGES ON DATABASE debian_versions TO app_user;  
\c my_db app_user  
GRANT ALL ON SCHEMA public TO app_user;
```



Принцип наименьших привилегий (Principle of Least Privilege — PoLP).

Бестпрактис настройки баз данных предполагают выдачу гранулярного доступа к объектам. То есть для пользователя создают учетную запись и дают ему права только на те таблицы или БД, которые ему действительно нужны, а не на все подряд.

5. Попробуйте подключиться к нашей БД под новым пользователем и проверить доступные действия (создание таблиц, создание БД и т.п.).
6. В отчет приведите консольный вывод таблицы в созданной БД с запросом, список всех пользователей с их правами.

Сетевой доступ

1. В *postgres.conf* измените *listen_addresses* так, чтобы сервер слушал не только *localhost*, но и внешний интерфейс. Сохраните измененную строку в отчет.
2. В *pg_hba.conf* добавьте правило доступа по IP (CIDR) с вашего компьютера или подсети, используйте метод *scram-sha-256*. Сохраните измененную строку в отчет.



IP-адрес будет меняться

Большинство персональных компьютеров не имеют собственного публичного IP-адреса. Вместо этого они находятся за сетевым экраном (NAT) вашего Wi-Fi роутера, и именно роутер использует внешний публичный IP-адрес для связи с интернетом. Следовательно, при подключении к другому Wi-Fi роутеру, айпи адрес «компьютера» сменится. Также сам Wi-Fi роутер, вероятно, имеет динамический белый IP, а не статический. Самый простой способ узнать свой белый IP - воспользоваться сайтами вроде <https://2ip.ru>.



Очень важно ограничивать доступ к инфраструктуре из интернета

Вывод инфраструктуры в интернет делает её легкой мишенью для недоброжелателей. Нужно четко понимать для чего делается вывод в интернет, какие порты открываются, с каких адресов будет идти трафик и т.п.

3. Установите на вашем компьютере *psql*

- Для Linux — пакет *postgresql-client*
 - Windows и MacOS (выберите установку только клиента) - <https://www.postgresql.org/download/> или, для windows <https://postgrespro.ru/windows>
 - или воспользуйтесь просмотрщиком БД в любой IDE JetBrains, если вы активировали студенческую подписку.
4. Подключитесь к БД с вашего компьютера машины (через *psql*). Сохраните команду подключения в отчет.

Подключение librespeed

1. Создайте для приложения отдельную БД.
2. Дайте вашему пользователю все права на эту БД.
3. Выполните:

```
SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SET check_function_bodies = false;
SET client_min_messages = warning;
SET row_security = off;

CREATE EXTENSION IF NOT EXISTS plpgsql WITH SCHEMA pg_catalog;

COMMENT ON EXTENSION plpgsql IS 'PL/pgSQL procedural language';

SET search_path = public, pg_catalog;
SET default_tablespace = '';
SET default_with_oids = false;

CREATE TABLE speedtest_users (
    id integer NOT NULL,
    "timestamp" timestamp without time zone DEFAULT now() NOT NULL,
    ip text NOT NULL,
    ispinfo text,
    extra text,
    ua text NOT NULL,
    lang text NOT NULL,
    dl text,
    ul text,
    ping text,
    jitter text,
    log text
);

CREATE SEQUENCE speedtest_users_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;

ALTER SEQUENCE speedtest_users_id_seq OWNED BY speedtest_users.id;

ALTER TABLE ONLY speedtest_users ALTER COLUMN id SET DEFAULT nextval('speedtest_users_id_seq'::regclass);

SELECT pg_catalog.setval('speedtest_users_id_seq', 1, true);

ALTER TABLE ONLY speedtest_users
    ADD CONSTRAINT speedtest_users_pkey PRIMARY KEY (id);
```

4. Создайте в корне приложения .env файл и введите туда переменные для подключения.
5. Поработайте с приложением, убедитесь, что данные стали писаться в БД.

Резервное копирование

1. Выполните логический бэкап базы приложения через *pg_dump* в формат SQL;
2. Создайте новую тестовую БД и восстановите в неё данные из SQL-дампа (через *psql*);
3. Напишите *bash* скрипт, который осуществляет резервное копирование вашей БД, в файл с именем формата YYYY-MM-DD-HH-MM-ИмяБД.sql, архивирует его и пишет в S3 бакет облака из предыдущей работы.
4. Проверьте работу скрипта. Добавьте его в отчет.

Вопросы

1. Перечислите основные бинарные пакеты (название и назначение), относящиеся к PostgreSQL и доступные в репозитории вашего Linux.
2. Если вам нужно установить последнюю версию СУБД, то как это можно сделать?
3. Что такое кластер PostgreSQL?
4. Какие базы данных создаются в PostgreSQL по умолчанию? Для чего они нужны?
5. Когда мы назначали права пользователю, мы давали права на БД и потом на схему. Зачем? Что дает каждое действие?
6. Почему опасно писать в `pg_hba.conf` строку с `0.0.0.0/0` и авторизацией `trust`?
7. Чем отличается `md5` от `scram-sha-256` в контексте безопасности?
8. Что из себя представлял бэкап базы?
9. Используя документацию (<https://postgrespro.ru/docs/postgresql/18>), сравните два упомянутых подхода к резервному копированию PostgreSQL по следующим показателям: формат, размер копии, скорость бэкапа, какие объекты можно отдельно восстановить из бэкапа, возможность PITR (Point-in-Time Recovery) — восстановления базы данных до произвольного момента времени в прошлом, возможность переноса бэкапов между разными версиями СУБД
10. Почему логический бэкап называют переносимым между разными версиями PostgreSQL и даже архитектурами?

Понятийный минимум

1. Основные архитектурные компоненты PostgreSQL
2. Возможности PostgreSQL как СУБД
3. Конфигурационные файлы PostgreSQL
4. Назначение и основные возможности утилиты `psql`
5. Виды резервного копирования в PostgreSQL